COS568: Learned Index — Milestone 1

Evaluation of B$^+$Tree, Dynamic PGM, and LIPP
on Lookup and Insertion Performance

Yuming Lou
yl1836@princeton.edu

March 29, 2026

1 Experimental Setup

Indexes. We evaluate three index structures:

- **B$^+$Tree**[2] — a classical balanced multi-level tree; all data resides in linked leaf nodes, internal nodes carry routing keys only.
- **DynamicPGM**[1] — a learned index based on Piecewise Linear Approximation (PLA) that supports dynamic insertions via a logarithmic multi-buffer strategy.
- **LIPP**[3] — a learned index that uses per-node linear regression to predict exact slot positions, resolving conflicts via child-node creation.

Datasets.

- **FB (100M)**: 100M 64-bit integer keys from Facebook data.
- **Books (100M)**: 100M 64-bit integer keys from Amazon book popularity data.
- **OSM-C (100M)**: 100M 64-bit integer keys from OpenStreetMap geographic coordinates.

Workloads. All workloads use 2 M total operations and a negative lookup ratio of 0.5 (half of lookup keys are absent from the index).

1. **Lookup-Only**: 1M positive + 1M negative lookups.
2. **Insert-Lookup (50/50)**: 1M insertions *then* 1M lookups; we report insert throughput and post-insert lookup throughput separately.
3. **Mixed 10% Insert**: insert and lookup operations interleaved, 10% inserts.
4. **Mixed 90% Insert**: insert and lookup operations interleaved, 90% inserts.

Hyperparameters. B$^+$Tree is tested with three (search method, max-node-logsize) combinations. DynamicPGM is tested with three (search method, error bound ε) combinations. LIPP has no hyperparameters. Each configuration is repeated **3 times**; all three run values are reported. The best-performing configuration per index is highlighted in green.

Cluster. I ran all the experiments on Della cluster, applying for 16 CPUs on 1 node and 64GB memory. I did not use any GPUs.

2 Index Size

Table 1: Index size in bytes (recorded from the lookup-only run, best config per index).

Index	FB (100M)	Books (100M)	OSM-C (100M)
LIPP	12.71 GB	11.83 GB	20.63 GB
B^+Tree	1.86 GB	1.86 GB	1.86 GB
DynamicPGM	1.72 GB	1.72 GB	1.72 GB

LIPP requires **7**–**11**× more memory than the tree-based indexes because it pre-allocates slot arrays with many NULL entries to absorb future conflict-free insertions without reallocation.

3 Workload 1: Lookup-Only

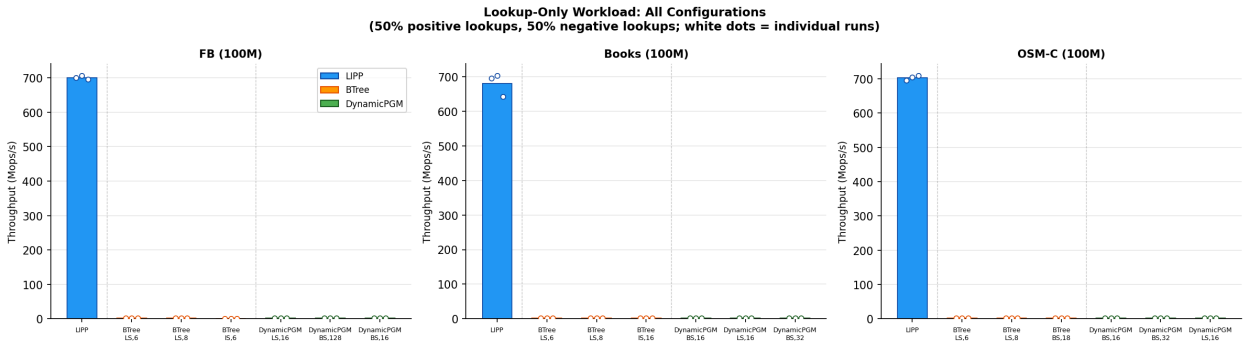

Figure 1: Lookup-only throughput (Mops/s) for all configurations across three datasets. White dots mark the three individual run values. Dashed vertical lines separate index families.

FB (100M)

Table 2: Lookup-Only throughput (Mops/s) — FB 100M.

Index	Config	Run 1	Run 2	Run 3	Mean	Std
LIPP	—	699.752	705.974	695.803	700.51	5.13
BTree	LinearSearch, size=6	1.573	1.615	1.613	1.600	0.023
BTree	LinearSearch, size=8	1.746	1.727	1.711	**1.728**	0.018
BTree	InterpolationSearch, size=6	1.299	1.315	1.295	1.303	0.011
DynamicPGM	LinearSearch, ε=16	2.477	2.452	2.448	**2.459**	0.016
DynamicPGM	BinarySearch, ε=128	1.748	1.800	1.795	1.781	0.028
DynamicPGM	BinarySearch, ε=16	2.325	2.288	2.283	2.299	0.023

Books (100M)

Table 3: Lookup-Only throughput (Mops/s) — Books 100M.

Index	Config	Run 1	Run 2	Run 3	Mean	Std
LIPP	—	696.021	702.404	641.913	680.11	33.24
BTree	LinearSearch, size=6	1.596	1.589	1.561	1.582	0.018
BTree	LinearSearch, size=8	1.726	1.711	1.700	**1.712**	0.013
BTree	InterpolationSearch, size=16	1.660	1.637	1.613	1.636	0.024
DynamicPGM	BinarySearch, ε=16	2.336	2.326	2.449	2.371	0.068
DynamicPGM	LinearSearch, ε=16	2.585	2.551	2.506	**2.547**	0.040
DynamicPGM	BinarySearch, ε=32	2.403	2.409	2.398	2.403	0.005

OSM-C (100M)

Table 4: Lookup-Only throughput (Mops/s) — OSM-C 100M.

Index	Config	Run 1	Run 2	Run 3	Mean	Std
LIPP	—	695.811	705.051	708.493	703.12	6.56
BTree	LinearSearch, size=6	2.124	2.115	2.135	2.125	0.010
BTree	LinearSearch, size=8	2.227	2.268	2.244	**2.247**	0.021
BTree	BinarySearch, size=18	1.790	1.749	1.804	1.781	0.029
DynamicPGM	BinarySearch, ε=16	2.624	2.658	2.704	**2.662**	0.040
DynamicPGM	BinarySearch, ε=32	2.511	2.563	2.548	2.541	0.027
DynamicPGM	LinearSearch, ε=16	2.564	2.531	2.606	2.567	0.038

4 Workload 2: Insert Throughput (50% Insert / 50% Lookup)

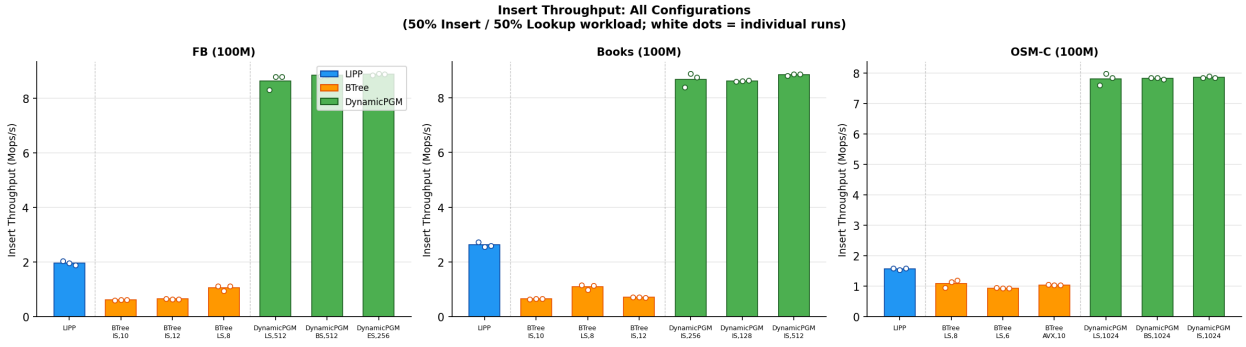

Figure 2: Insert throughput (Mops/s) for all configurations. 1M insertions are performed first; throughput is measured over this phase.

FB (100M)

Table 5: Insert throughput (Mops/s) — FB 100M.

Index	Config	Run 1	Run 2	Run 3	Mean	Std
LIPP	—	2.047	1.962	1.895	1.968	0.076
BTree	InterpolationSearch, size=10	0.600	0.631	0.633	0.621	0.019
BTree	InterpolationSearch, size=12	0.657	0.652	0.652	0.653	0.003
BTree	LinearSearch, size=8	1.124	0.951	1.114	**1.063**	0.097
DynamicPGM	LinearSearch, ε=512	8.320	8.792	8.806	8.639	0.277
DynamicPGM	BinarySearch, ε=512	8.820	8.852	8.892	8.855	0.036
DynamicPGM	ExponentialSearch, ε=256	8.849	8.906	8.899	**8.885**	0.031

Books (100M)

Table 6: Insert throughput (Mops/s) — Books 100M.

Index	Config	Run 1	Run 2	Run 3	Mean	Std
LIPP	—	2.734	2.555	2.597	**2.629**	0.094
BTree	InterpolationSearch, size=10	0.638	0.657	0.667	0.654	0.015
BTree	LinearSearch, size=8	1.160	0.982	1.147	**1.096**	0.099
BTree	InterpolationSearch, size=12	0.712	0.726	0.705	0.714	0.011
DynamicPGM	InterpolationSearch, ε=256	8.385	8.874	8.738	8.665	0.252
DynamicPGM	InterpolationSearch, ε=128	8.589	8.603	8.638	8.610	0.025
DynamicPGM	InterpolationSearch, ε=512	8.809	8.855	8.856	**8.840**	0.027

OSM-C (100M)

Table 7: Insert throughput (Mops/s) — OSM-C 100M.

Index	Config	Run 1	Run 2	Run 3	Mean	Std
LIPP	—	1.586	1.537	1.597	1.574	0.032
BTree	LinearSearch, size=8	0.950	1.148	1.189	1.096	0.128
BTree	LinearSearch, size=6	0.946	0.939	0.936	0.940	0.006
BTree	LinearAVX, size=10	1.052	1.045	1.044	**1.047**	0.004
DynamicPGM	LinearSearch, ε=1024	7.600	7.977	7.848	7.808	0.192
DynamicPGM	BinarySearch, ε=1024	7.851	7.843	7.791	7.829	0.032
DynamicPGM	InterpolationSearch, ε=1024	7.842	7.900	7.851	**7.864**	0.031

5 Workload 2 (cont.): Lookup Throughput After Insertions

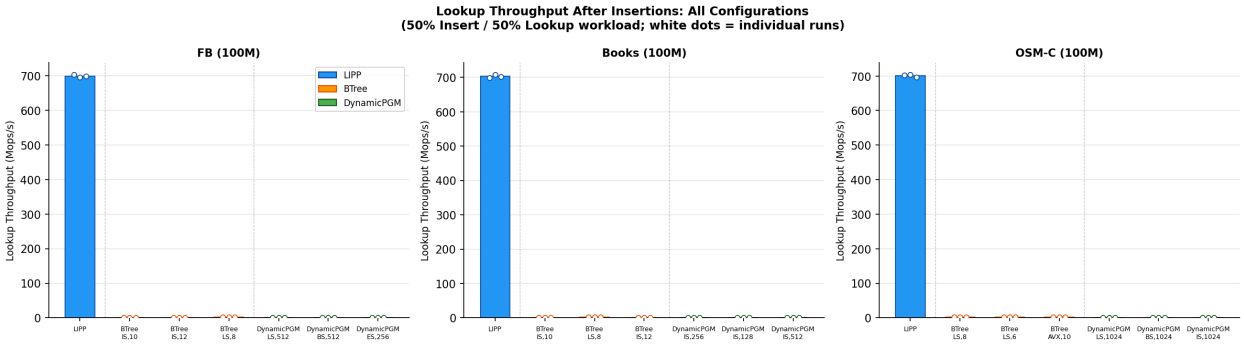

Figure 3: Lookup throughput (Mops/s) measured *after* 1M insertions have been performed.

FB (100M)

Table 8: Post-insert lookup throughput (Mops/s) — FB 100M.

Index	Config	Run 1	Run 2	Run 3	Mean	Std
LIPP	—	702.574	695.281	698.389	698.75	3.66
BTree	InterpolationSearch, size=10	1.012	1.037	1.000	1.016	0.019
BTree	InterpolationSearch, size=12	0.991	0.986	0.999	0.992	0.006
BTree	LinearSearch, size=8	1.490	1.493	1.479	**1.487**	0.007
DynamicPGM	LinearSearch, ε=512	0.371	0.375	0.375	0.374	0.002
DynamicPGM	BinarySearch, ε=512	0.670	0.654	0.655	**0.659**	0.009
DynamicPGM	ExponentialSearch, ε=256	0.658	0.645	0.661	0.655	0.008

Books (100M)

Table 9: Post-insert lookup throughput (Mops/s) — Books 100M.

Index	Config	Run 1	Run 2	Run 3	Mean	Std
LIPP	—	699.463	707.440	701.909	702.94	4.09
BTree	InterpolationSearch, size=10	1.067	1.102	1.113	1.094	0.024
BTree	LinearSearch, size=8	1.500	1.521	1.517	**1.513**	0.011
BTree	InterpolationSearch, size=12	1.110	1.147	1.128	1.128	0.018
DynamicPGM	InterpolationSearch, ε=256	0.623	0.628	0.607	0.619	0.011
DynamicPGM	InterpolationSearch, ε=128	0.598	0.594	0.615	0.603	0.011
DynamicPGM	InterpolationSearch, ε=512	0.618	0.621	0.637	**0.625**	0.010

OSM-C (100M)

Table 10: Post-insert lookup throughput (Mops/s) — OSM-C 100M.

Index	Config	Run 1	Run 2	Run 3	Mean	Std
LIPP	—	701.904	703.474	696.128	700.50	3.87
BTree	LinearSearch, size=8	1.908	2.011	2.067	**1.995**	0.081
BTree	LinearSearch, size=6	1.818	1.794	1.793	1.802	0.014
BTree	LinearAVX, size=10	1.824	1.825	1.759	1.803	0.038
DynamicPGM	LinearSearch, ε=1024	0.178	0.179	0.179	0.178	0.001
DynamicPGM	BinarySearch, ε=1024	0.719	0.724	0.720	**0.721**	0.003
DynamicPGM	InterpolationSearch, ε=1024	0.395	0.403	0.397	0.398	0.004

6 Workload 3: Mixed (10% Insert, 90% Lookup)

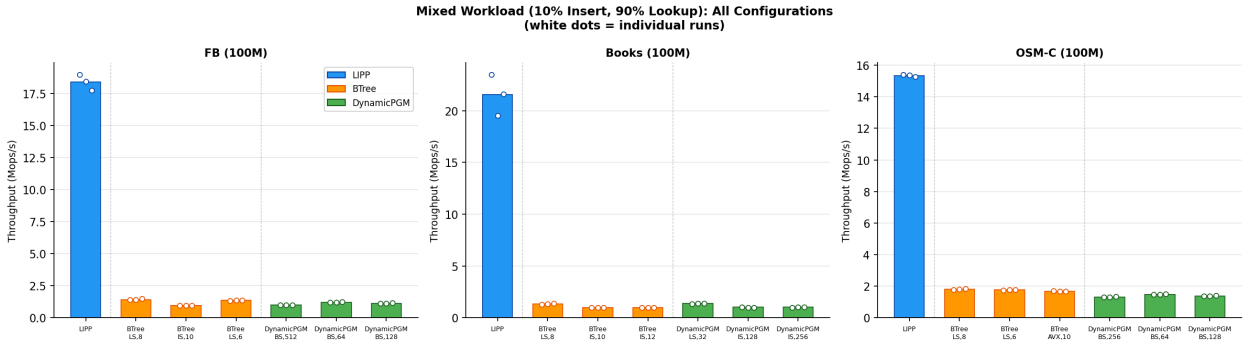

Figure 4: Mixed throughput (Mops/s) for the 10% insert / 90% lookup interleaved workload.

FB (100M)

Table 11: Mixed 10% insert throughput (Mops/s) — FB 100M.

Index	Config	Run 1	Run 2	Run 3	Mean	Std
LIPP	—	18.977	18.479	17.747	18.401	0.619
BTree	LinearSearch, size=8	1.399	1.393	1.488	**1.426**	0.053
BTree	InterpolationSearch, size=10	0.960	0.958	0.956	0.958	0.002
BTree	LinearSearch, size=6	1.340	1.373	1.388	1.367	0.025
DynamicPGM	BinarySearch, ε=512	1.000	0.989	0.988	0.992	0.007
DynamicPGM	BinarySearch, ε=64	1.223	1.213	1.239	**1.225**	0.013
DynamicPGM	BinarySearch, ε=128	1.139	1.142	1.151	1.144	0.006

Books (100M)

Table 12: Mixed 10% insert throughput (Mops/s) — Books 100M.

Index	Config	Run 1	Run 2	Run 3	Mean	Std
LIPP	—	23.461	19.535	21.616	21.537	1.964
BTree	LinearSearch, size=8	1.312	1.362	1.400	**1.358**	0.044
BTree	InterpolationSearch, size=10	0.990	0.979	0.970	0.980	0.010
BTree	InterpolationSearch, size=12	0.984	0.980	0.994	0.986	0.008
DynamicPGM	LinearSearch, ε=32	1.362	1.368	1.365	**1.365**	0.003
DynamicPGM	InterpolationSearch, ε=128	1.019	1.008	1.008	1.012	0.006
DynamicPGM	InterpolationSearch, ε=256	0.983	1.045	1.026	1.018	0.032

OSM-C (100M)

Table 13: Mixed 10% insert throughput (Mops/s) — OSM-C 100M.

Index	Config	Run 1	Run 2	Run 3	Mean	Std
LIPP	—	15.395	15.369	15.265	15.343	0.069
BTree	LinearSearch, size=8	1.763	1.793	1.828	**1.795**	0.033
BTree	LinearSearch, size=6	1.738	1.775	1.770	1.761	0.020
BTree	LinearAVX, size=10	1.708	1.684	1.670	1.687	0.019
DynamicPGM	BinarySearch, ε=256	1.296	1.311	1.335	1.314	0.020
DynamicPGM	BinarySearch, ε=64	1.462	1.478	1.498	**1.480**	0.018
DynamicPGM	BinarySearch, ε=128	1.385	1.381	1.397	1.388	0.009

7 Workload 4: Mixed (90% Insert, 10% Lookup)

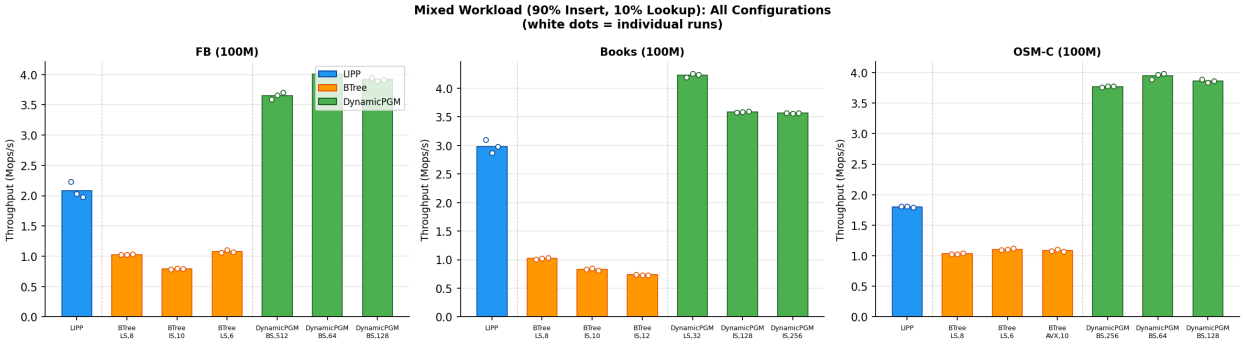

Figure 5: Mixed throughput (Mops/s) for the 90% insert / 10% lookup interleaved workload.

FB (100M)

Table 14: Mixed 90% insert throughput (Mops/s) — FB 100M.

Index	Config	Run 1	Run 2	Run 3	Mean	Std
LIPP	—	2.228	2.032	1.976	2.078	0.133
BTree	LinearSearch, size=8	1.029	1.022	1.035	1.029	0.006
BTree	InterpolationSearch, size=10	0.783	0.802	0.791	0.792	0.010
BTree	LinearSearch, size=6	1.058	1.102	1.065	**1.075**	0.024
DynamicPGM	BinarySearch, ε=512	3.590	3.658	3.701	3.649	0.056
DynamicPGM	BinarySearch, ε=64	3.997	4.008	4.011	**4.006**	0.007
DynamicPGM	BinarySearch, ε=128	3.948	3.892	3.913	3.917	0.028

Books (100M)

Table 15: Mixed 90% insert throughput (Mops/s) — Books 100M.

Index	Config	Run 1	Run 2	Run 3	Mean	Std
LIPP	—	3.100	2.866	2.978	2.981	0.117
BTree	LinearSearch, size=8	1.003	1.025	1.034	**1.021**	0.016
BTree	InterpolationSearch, size=10	0.834	0.846	0.809	0.830	0.019
BTree	InterpolationSearch, size=12	0.742	0.733	0.733	0.736	0.006
DynamicPGM	LinearSearch, ε=32	4.189	4.254	4.239	**4.227**	0.034
DynamicPGM	InterpolationSearch, ε=128	3.578	3.586	3.597	3.587	0.010
DynamicPGM	InterpolationSearch, ε=256	3.568	3.561	3.571	3.566	0.005

OSM-C (100M)

Table 16: Mixed 90% insert throughput (Mops/s) — OSM-C 100M.

Index	Config	Run 1	Run 2	Run 3	Mean	Std
LIPP	—	1.807	1.809	1.794	1.803	0.008
BTree	LinearSearch, size=8	1.027	1.030	1.045	1.034	0.009
BTree	LinearSearch, size=6	1.098	1.102	1.124	**1.108**	0.014
BTree	LinearAVX, size=10	1.078	1.106	1.073	1.086	0.018
DynamicPGM	BinarySearch, ε=256	3.762	3.779	3.774	3.772	0.009
DynamicPGM	BinarySearch, ε=64	3.888	3.966	3.980	**3.945**	0.050
DynamicPGM	BinarySearch, ε=128	3.885	3.833	3.863	3.860	0.026

8 Summary Comparison (Best Configuration per Index)

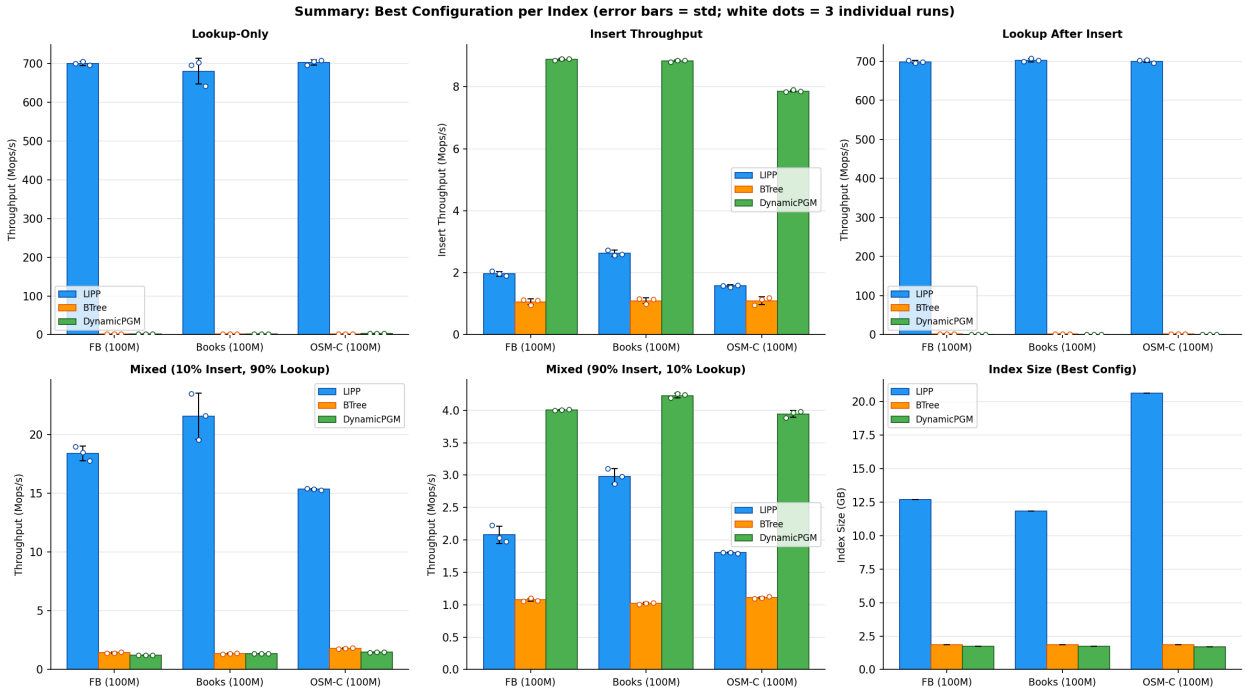

Figure 6: Best-configuration summary across all workloads and datasets. Error bars show std over 3 runs; white dots show individual run values. Bottom-right panel shows index size (GB).

Table 17: Best-configuration mean throughput (Mops/s) per workload and dataset. "Ins." = insert throughput; "Lkp.@Ins." = lookup throughput after insertions; "Mix10" = mixed 10% insert; "Mix90" = mixed 90% insert.

Dataset	Index	Lookup-Only	Ins.	Lkp.@Ins.	Mix10	Mix90
FB (100M)	LIPP	700.51	1.97	698.75	18.40	2.08
	B$^+$Tree	1.73	1.06	1.49	1.43	1.08
	DynamicPGM	2.46	8.89	0.66	1.23	4.01
Books (100M)	LIPP	680.11	2.63	702.94	21.54	2.98
	B$^+$Tree	1.71	1.10	1.51	1.36	1.02
	DynamicPGM	2.55	8.84	0.63	1.37	4.23
OSM-C (100M)	LIPP	703.12	1.57	700.50	15.34	1.80
	B$^+$Tree	2.25	1.10	2.00	1.80	1.11
	DynamicPGM	2.66	7.86	0.72	1.48	3.95

9 Analysis: Why These Differences?

9.1 Why is LIPP's Lookup so Dominant?

LIPP achieves **270**–**460**× higher pure lookup throughput than B$^+$Tree and DynamicPGM (≈ 700 vs. $\approx$ 1.7–2.7 Mops/s). Three factors explain this:

1. **Zero-search intra-node access.** Each LIPP node stores a linear regression that maps a key to an *exact* slot index in the node's array. Lookup at every level is a single arithmetic computation followed by one array dereference—no binary search or interpolation is needed within the node.
2. **Cache-friendliness.** LIPP stores data in contiguous slot arrays. A traversal from root to leaf touches only $O(h)$ cache lines (where h is the tree height), with no pointer-chasing to non-contiguous B-tree nodes or segment lists.
3. **No false-positive cost for negative lookups.** When a key is absent, the predicted slot simply does not match the stored key; the lookup terminates in $O(1)$ per level with no additional search.

The same advantages persist after insertions: LIPP's post-insertion lookup throughput ($\approx$ 699–703 Mops/s) is nearly identical to the pure lookup case because each new key either fills a NULL slot (no structural change) or creates a child node that contains its own fitted linear model—neither case invalidates the existing models.

9.2 Why is DynamicPGM's Insert so Fast?

DynamicPGM achieves **3**–**6**× higher insert throughput than LIPP and **7**–**9**× more than B$^+$Tree ($\approx$ 8–9 vs. $\approx$ 1–2.6 Mops/s).

The key mechanism is the **logarithmic method**: DynamicPGM maintains a set of PGM sub-indexes of exponentially increasing sizes $\{2^0, 2^1, \ldots, 2^{\lceil \log n \rceil}\}$. Inserting a new key costs $O(1)$ amortized—it appends to the smallest buffer, and a merge of two buffers of size k (costing $O(k \log k)$) is amortized across the k insertions that caused it. Crucially, *no node split propagates upward* on each insert, unlike B$^+$Tree, and *no model is re-trained on the critical path.*

By contrast, B$^+$Tree must find the correct leaf and potentially split nodes up the tree on each insertion, incurring multiple cache misses and pointer updates.

LIPP's insert is slower because each insertion may trigger a *conflict*: if the linear model maps the new key to an already-occupied DATA slot, LIPP must create a new child node and refit that node's model—an $O(n_{\text{node}})$ operation that becomes more frequent as the index grows.

9.3 Why Does DynamicPGM's Lookup Degrade After Insertions?

After M insertions, the logarithmic method produces $O(\log M)$ non-empty sub-indexes. A lookup must sequentially search *every* sub-index until the key is found (or all are exhausted for negative keys). Each sub-index search costs $O(\log \varepsilon)$ per level, so the total lookup cost rises to $O(\log M \cdot \log \varepsilon)$—substantially worse than the pure static case.

This explains the dramatic drop visible in the data: on FB, DynamicPGM post-insert lookup throughput falls from $\approx$ 2.46 Mops/s (lookup-only) to $\approx$ 0.66 Mops/s (after 1M insertions), a **3.7**×

slowdown. LIPP shows essentially *no* such degradation because its model-guided access remains $O(h)$ regardless of how many keys have been inserted.

9.4 Why is B$^+$Tree Consistently the Slowest?

B$^+$Tree cannot exploit data distribution at all. Lookups traverse $O(\log_B N)$ levels, each requiring a local search (linear or interpolation) within a node and a pointer dereference to the next level, causing $O(\log_B N)$ cache misses. Insertions similarly incur $O(\log_B N)$ node accesses and may trigger splits. The tree is fully general-purpose: without any learned model, it cannot shortcut navigation or avoid rebalancing.

9.5 Hyperparameter Sensitivity

For **B$^+$Tree**, the node-size parameter has a moderate effect. Larger nodes reduce tree height (fewer levels to traverse) but increase intra-node search cost; the optimum across all workloads is consistently `LinearSearch, size=8`, which balances these factors.

For **DynamicPGM**, the error bound ε controls how many keys one linear segment covers. A larger ε means fewer segments (faster bulk loading, fewer level transitions) but longer local binary searches at query time. For lookup-heavy workloads, smaller ε (e.g., 16–32) wins; for insert-heavy mixed workloads, larger ε (64–512) reduces merge overhead and is optimal.

9.6 Dataset-Specific Observations

OSM-C gives slightly higher DynamicPGM lookup throughput than FB/Books (2.66 vs. 2.46–2.55 Mops/s), consistent with geographic coordinates having a more uniform distribution that is better approximated by fewer PLA segments.

LIPP on Books shows higher variance across 3 runs (std = 33.2 vs. ≤ 6.6 on the other datasets), likely reflecting OS-level interference or NUMA effects during one run. The minimum run value (641.9 Mops/s) is still $\approx 250\times$ faster than B$^+$Tree.

10 Conclusion

The three indexes show strongly complementary performance profiles:

- **LIPP** achieves exceptional lookup speed ($\approx$ 700 Mops/s, 300–460$\times$ faster than competitors) by replacing intra-node search with exact slot prediction. Lookup performance is fully preserved after insertions. However, LIPP uses 7–11$\times$ more memory and inserts slower under conflict-heavy workloads.
- **DynamicPGM** delivers the highest insert throughput ($\approx$ 8–9 Mops/s, 3–6$\times$ faster than LIPP) via amortized logarithmic-method merging. It is the best choice for insert-heavy mixed workloads (90% insert). However, its post-insert lookup throughput degrades by 3–4$\times$ due to the multi-index sequential scan.
- **B$^+$Tree** is the most predictable index but the slowest in every scenario: it has no distribution awareness, traverses $O(\log_B N)$ cache-unfriendly pointer-chased levels, and cannot avoid node splits during insertions.

These complementary strengths motivate the *Hybrid DynamicPGM + LIPP* design explored in Milestones 2 and 3: use DynamicPGM's fast insert path for incremental updates, then periodically flush the accumulated buffer into LIPP to restore lookup efficiency.

References

[1] Paolo Ferragina and Giorgio Vinciguerra. The pgm-index: a fully-dynamic compressed learned index with provable worst-case bounds. *Proc. VLDB Endow.*, 13(8):1162–1175, April 2020.

[2] CS 186 Course Staff. Course note 4: Buffer Management. https://cs186berkeley.net/notes/note4/, 2024.

[3] Jiacheng Wu, Yong Zhang, Shimin Chen, Jin Wang, Yu Chen, and Chunxiao Xing. Updatable learned index with precise positions, 2021.